

ALGORITMI E STRUTTURE DATI

ALGORITMI DI RICERCA + STRUTTURE DATI

AULA VIRTUALE DEL 14-05-2026

V 1.0

ALGORITMI DI RICERCA

Un **algoritmo di ricerca** serve a trovare un elemento all'interno di una struttura dati, per esempio un array o una lista. Il problema tipico è:

dato un insieme di elementi e un valore da cercare, stabilire se il valore è presente e, se sì, in quale posizione. Esempio:
array A = [7, 3, 9, 12, 5]
valore cercato x = 12
risultato: 12 è presente in posizione 3, usando indici che partono da 0.

Gli algoritmi di ricerca più semplici e importanti sono **Linear Search** e **Binary Search**.

ALGORITMI DI RICERCA: LINEAR SEARCH

La Ricerca Lineare (o Linear Search) controlla gli elementi uno alla volta, partendo dal primo fino a trovare quello cercato oppure fino alla fine della struttura dati. Esempio, dato un array

A = [4, 8, 15, 16, 23, 42]

e un valore da cercare:

x = 16

l'algoritmo confronta:

4 == 16? no

8 == 16? no

15 == 16? no

16 == 16? sì

A questo punto si ferma e restituisce la posizione dell'elemento.

ALGORITMI DI RICERCA: COMPLESSITA' LINEAR SEARCH

Nel caso migliore, l'elemento cercato è il primo: basta un solo confronto. T(n) = O(1)

Nel caso peggiore, l'elemento è l'ultimo oppure non è presente: bisogna controllare tutti gli n elementi. T(n) = O(n)

Nel caso medio, in media si controlla circa metà array, quindi la complessità resta lineare: T(n) = O(n)

Vantaggi
La ricerca lineare è semplice, funziona su array ordinati e non ordinati, e non richiede alcuna preparazione dei dati.
Svantaggi
Diventa inefficiente quando il numero di elementi è grande, perché nel caso peggiore deve controllare tutta la struttura dati.

ALGORITMI DI RICERCA: BINARY SEARCH

La ricerca binaria è più efficiente, ma richiede una condizione fondamentale: i dati devono essere ordinati. Esempio, dato un array ordinato:
A = [3, 7, 12, 18, 25, 31, 40]
e un valore da cercare:
x = 25
l'algoritmo guarda l'elemento centrale.

Elemento centrale: 18
Poiché 25 > 18, non serve cercare nella metà sinistra. Si continua solo nella metà destra:
[25, 31, 40]
Nuovo elemento centrale: 31
Poiché 25 < 31, si cerca nella metà sinistra: [25]

Elemento trovato.
L'idea centrale è che a ogni passo si elimina metà dello spazio di ricerca.

ALGORITMI DI RICERCA: COMPLESSITA' BINARY SEARCH

Nel caso migliore, l'elemento cercato è subito al centro:
$T(n) = O(1)$
Nel caso peggiore, a ogni passo l'array viene dimezzato. Dopo un certo numero di dimezzamenti si arriva a un solo elemento. Per questo la complessità è:
$T(n) = O(\log n)$
Anche nel caso medio la complessità è:
$T(n) = O(\log n)$

Vantaggi
La ricerca binaria è molto più veloce della ricerca lineare su grandi quantità di dati.
Per esempio, con circa un milione di elementi, la ricerca lineare può richiedere fino a un milione di confronti, mentre la ricerca binaria ne richiede circa 20.
Svantaggi
Richiede che i dati siano ordinati. Inoltre, è particolarmente adatta a strutture con accesso diretto agli elementi, come gli array. Su una lista concatenata, invece, raggiungere l'elemento centrale non è immediato, quindi perde gran parte del suo vantaggio.

ALGORITMI DI RICERCA: CONFRONTO

Quando scegliere l'una o l'altra?

Usiamo la **ricerca lineare** quando i dati non sono ordinati, quando la struttura è piccola, oppure quando l'ordinamento costerebbe più della ricerca stessa.

Usiamo la **ricerca binaria** quando i dati sono ordinati, quando dobbiamo effettuare molte ricerche, oppure quando lavoriamo con array grandi e vogliamo ridurre il numero di confronti.

Un **punto importante** è questo: ordinare i dati ha un costo. Se dobbiamo fare una sola ricerca su pochi elementi, può convenire la ricerca lineare. Se invece dobbiamo fare molte ricerche sugli stessi dati, può convenire ordinare prima i dati e poi usare la ricerca binaria.

ALGORITMI DI RICERCA: CONFRONTO

Confronto: ricerca lineare vs ricerca binaria

Aspetto	Ricerca lineare	Ricerca binaria
Dati ordinati necessari?	No	Sì
Strategia	Controlla un elemento alla volta	Divide lo spazio di ricerca a metà
Caso migliore	`O(1)`	`O(1)`
Caso medio	`O(n)`	`O(log n)`
Caso peggiore	`O(n)`	`O(log n)`
Struttura ideale	Array o lista	Array ordinato
Semplicità	Molto semplice	Leggermente più complessa
Quando usarla	Pochi dati o dati non ordinati	Molti dati ordinati

ALGORITMI DI RICERCA: LINEAR SEARCH

```
def ricerca_lineare(A, x):
    """
    Cerca il valore x all'interno della lista A.
    Restituisce l'indice dell'elemento se trovato,
    altrimenti restituisce -1.
    """

    # Scorriamo tutti gli elementi della lista, uno alla volta
    for i in range(len(A)):

        # Confrontiamo l'elemento corrente con il valore cercato
        if A[i] == x:

            # Se l'elemento è stato trovato, restituiamo la sua posizione
            return i

    # Se arriviamo qui, significa che x non è presente nella lista
    return -1
```

ALGORITMI DI RICERCA: LINEAR SEARCH

```
# Esempio di utilizzo
A = [4, 8, 15, 16, 23, 42]
x = 16

posizione = ricerca_lineare(A, x)

if posizione != -1:
    print(f"Elemento {x} trovato in posizione {posizione}")
else:
    print(f"Elemento {x} non trovato")
```

ALGORITMI DI RICERCA: BINARY SEARCH

```
def binary_search(lista_ordinata, valore_cercato):
    """
    Cerca un valore in una lista ordinata usando la ricerca binaria.

    Parametri:
    lista_ordinata: lista ordinata in modo crescente
    valore_cercato: elemento da trovare

    Ritorna:
    indice dell'elemento se trovato, altrimenti -1
    """

    # Estremo sinistro dell'intervallo di ricerca
    sinistra = 0

    # Estremo destro dell'intervallo di ricerca
    destra = len(lista_ordinata) - 1

    # Continuiamo finché l'intervallo è valido
    while sinistra <= destra:

        # Calcoliamo l'indice centrale
        centro = (sinistra + destra) // 2
```

ALGORITMI DI RICERCA: BINARY SEARCH

```
        # Se l'elemento centrale è quello cercato, abbiamo finito
        if lista_ordinata[centro] == valore_cercato:
            return centro

        # Se il valore cercato è più grande,
        # dobbiamo cercare nella metà destra
        elif lista_ordinata[centro] < valore_cercato:
            sinistra = centro + 1

        # Se il valore cercato è più piccolo,
        # dobbiamo cercare nella metà sinistra
        else:
            destra = centro - 1

    # Se l'intervallo si esaurisce, il valore non è presente
    return -1

# Esempio di utilizzo
numeri = [1, 3, 6, 8, 10, 14]
valore = 10

risultato = binary_search(numeri, valore)

if risultato != -1:
    print(f"Valore {valore} trovato in posizione {risultato}")
else:
    print(f"Valore {valore} non trovato")
```

ALGORITMI DI RICERCA: BINARY SEARCH RICORSIVA

```
def binary_search_ricorsiva(lista_ordinata, valore_cercato, sinistra, destra):
    """
    Cerca un valore in una lista ordinata usando la ricerca binaria ricorsiva.

    Parametri:
    lista_ordinata: lista ordinata in modo crescente
    valore_cercato: elemento da trovare
    sinistra: indice iniziale dell'intervallo
    destra: indice finale dell'intervallo

    Ritorna:
    indice dell'elemento se trovato, altrimenti -1
    """

    # Caso base: intervallo vuoto
    if sinistra > destra:
        return -1

    # Calcoliamo l'indice centrale
    centro = (sinistra + destra) // 2

    # Caso base: valore trovato
    if lista_ordinata[centro] == valore_cercato:
        return centro
```

ALGORITMI DI RICERCA: BINARY SEARCH RICORSIVA

```
    # Se il valore cercato è maggiore, cerchiamo nella metà destra
    elif lista_ordinata[centro] < valore_cercato:
        return binary_search_ricorsiva(
            lista_ordinata,
            valore_cercato,
            centro + 1,
            destra
        )
    # Altrimenti cerchiamo nella metà sinistra
    else:
        return binary_search_ricorsiva(
            lista_ordinata,
            valore_cercato,
            sinistra,
            centro - 1
        )

# Esempio di utilizzo
numeri = [2, 5, 7, 9, 12, 15, 20]
valore = 7

risultato = binary_search_ricorsiva(numeri, valore, 0, len(numeri) - 1)

if risultato != -1:
    print(f"Valore {valore} trovato in posizione {risultato}")
else:
    print(f"Valore {valore} non trovato")
```

ALGORITMI DI RICERCA: CONFRONTO IN PYTHON

```
def linear_search_con_confronti(lista, valore_cercato):
    """
    Ricerca lineare con conteggio dei confronti.
    """

    confronti = 0

    for indice in range(len(lista)):
        confronti += 1

        if lista[indice] == valore_cercato:
            return indice, confronti

    return -1, confronti


def binary_search_con_confronti(lista_ordinata, valore_cercato):
    """
    Ricerca binaria con conteggio dei confronti.
    """

    sinistra = 0
    destra = len(lista_ordinata) - 1
    confronti = 0
```

ALGORITMI DI RICERCA: BINARY SEARCH RICORSIVA

```
    while sinistra <= destra:
        centro = (sinistra + destra) // 2
        confronti += 1

        if lista_ordinata[centro] == valore_cercato:
            return centro, confronti

        elif lista_ordinata[centro] < valore_cercato:
            sinistra = centro + 1

        else:
            destra = centro - 1

    return -1, confronti

# Lista ordinata da 1 a 100
numeri = list(range(1, 101))

# Cerchiamo un valore vicino alla fine
valore = 99

indice_lineare, confronti_lineare = linear_search_con_confronti(numeri, valore)
indice_binaria, confronti_binaria = binary_search_con_confronti(numeri, valore)

print("Ricerca lineare")
print(f"Indice trovato: {indice_lineare}")
print(f"Numero di confronti: {confronti_lineare}")
```

STRUTTURE DATI ELEMENTARI

In questa sezione approfondiremo le strutture dati elementari basate su strutture dati indicizzate (array) e quelle basata su strutture collegate (record e puntatori). In particolare:

- Array
- Pile e code
- Alberi

STRUTTURE DATI ELEMENTARI: ARRAY

Un array è una **struttura dati indicizzata** costituita da una collezione di celle numerate che possono contenere elementi di un tipo prestabilito. Assumeremo che in un array di dimensione h (contenente h celle) gli indici delle celle possano essere compresi indifferentemente tra 0 e h-1, oppure tra 1 e h (a seconda del linguaggio di programmazione utilizzato può essere l'uno o l'altro). Inoltre, assumeremo che il tempo in lettura e scrittura a una qualsiasi cella di un array sia un tempo costante.

STRUTTURE DATI ELEMENTARI: ARRAY

Effettueremo, inoltre, le seguenti assunzioni (espresso in termini di proprietà forti o deboli):

- Gli indici delle celle di un array sono numeri consecutivi (proprietà forte);
- Non è possibile aggiungere nuove celle ad un array (proprietà debole).

STRUTTURE DATI ELEMENTARI: ARRAY

La proprietà debole implica che il ridimensionamento di un array è possibile solo mediante riallocazione, cioè creando un nuovo array con la dimensione voluta, e copiando il contenuto delle celle di interesse dal vecchio array al nuovo array.

Questo richiede tempo proporzionale al minimo fra la vecchia e la nuova dimensione.

STRUTTURE DATI ELEMENTARI: ARRAY

In questo ambito, introduciamo quello che è il concetto di Dizionario, che non è altro una collezione di elementi a cui sono associate chiavi prese da un dominio totalmente ordinato. Operazioni tipiche su un Dizionario sono: inserimento (insert) di un elemento e di una chiave ad esso associata, cancellazione (delete) di un elemento data una chiave, ricerca (search) dell'elemento associato ad una data chiave.

STRUTTURE DATI ELEMENTARI: ARRAY

Nella rappresentazione di un dizionario con un array utilizzeremo la tecnica del raddoppiamento-dimezzamento. Questa tecnica consente di mantenere efficientemente in un array una collezione non ordinata di n elementi soggetta a operazioni di inserimento e cancellazione. La tecnica in questione è chiamata tecnica del raddoppiamento-dimezzamento (doubling-halving) ed in sostanza mantiene un array di dimensione h, dove per ogni n > , h soddisfa la seguente invariante: n <= h < 4n

STRUTTURE DATI ELEMENTARI: ARRAY

Le prime n celle dell'array contengono gli elementi della collezione, mentre il contenuto delle altre celle è indefinito. L'idea principale è quella di non effettuare riallocazioni ad ogni inserimento/cancellazione, ma solo ogni $\Omega(n)$ operazioni. L'invariante $n <= h <= 4n$ sulla dimensione dell'array viene mantenuta mediante riallocazioni, secondo i seguenti passi:

- Inizialmente , quando n=0, poniamo h=1
- Ogni qualvolta n supera h, l'array viene riallocato raddoppiandone la dimensione ($h \leftarrow 2h$)
- Ogni qualvolta n scende a h/4, l'array viene riallocato dimezzandone la dimensione ($h \leftarrow h/2$)

STRUTTURE DATI ELEMENTARI: ARRAY

In questo modo, per ogni n>0 si mantiene invariante: h/4 < n <= h , ovvero n <= h <= 4n. Lo spazio utilizzato per memorizzare la collezione è S(n)=Θ(h)= Θ(n).

STRUTTURE DATI ELEMENTARI: ARRAY

```
# ──────────────────────────────────────────────────────────
# 1. ARRAY
# ──────────────────────────────────────────────────────────
print("=" * 60)
print("1. ARRAY")
print("=" * 60)
arr = [10, 20, 30, 40, 50]

# Accesso O(1)
print(f"Elemento in posizione 2 : {arr[2]}")
# Inserimento in coda O(1) ammortizzato
arr.append(60)
print(f"Dopo append(60)         : {arr}")

# Inserimento in posizione i O(n)
arr.insert(2, 99)
print(f"Dopo insert(2, 99)      : {arr}")

# Rimozione per valore O(n)
arr.remove(99)
print(f"Dopo remove(99)         : {arr}")

# Ricerca O(n)
print(f"Indice di 40            : {arr.index(40)}")

# Slicing O(k)
print(f"Slice [1:4]             : {arr[1:4]}")
```

STRUTTURE DATI ELEMENTARI COLLEGATE: RECORD E PUNTATORI

Un altro modo per rappresentare collezioni di elementi è quella basata su strutture dati collegate. Per queste ultime possiamo citare i record, che, come le celle degli array, sono numerati e contengono ciascuno un oggetto della collezione. Mentre la numerazione delle celle degli arrayè locale al singolo array, i numeri associati ai record sono tipicamente i loro indirizzi in memoria, e quindi sono globali nell'ambito di un programma. Quindi, diversamente dalle celle degli array, che vengono create contestualmente all'array che le contiene, i record sono creati e distrutti individualmente ed in maniera dinamica nella memoria del calcolatore (quindi non necessariamente consecutivi).

STRUTTURE DATI ELEMENTARI COLLEGATE: RECORD E PUNTATORI

Per mantenere la relazioni tra record, ognuno di essi contiene uno o più indirizzi di altri record della collezione (collegamento). Se un record A contiene l'indirizzo di un altro record B, diremo che esiste un collegamento tra A e B realizzato tramite un puntatore.

- E' possibile aggiungere o togliere record a una struttura collegata (proprietà forte);
- Gli indirizzi dei record di una struttura collegata non sono necessariamente consecutivi (proprietà debole).

STRUTTURE DATI ELEMENTARI: PILE E CODE

Nella vita reale, ci troviamo spesso a gestire insiemi di oggetti. Si pensi quando raccogliamo i piatti dopo una cena, li impiliamo formando una catasta e se li poggiamo nel lavandino, li prendiamo nell'ordine opposto a quello in cui li abbiamo impilati: l'ultimo piatto depositato sulla pila sarà il primo ad essere lavato. Altro scenario è quello delle code di attesa, per esempio gli utenti di un ufficio postale, il primo arrivato è il primo ad essere servito. Possiamo formalizzare questo in:

- LIFO (last-in-first-out): per questo accesso definiamo la Pila;
- FIFO (first-in-first-out): per questo accesso definiamo la Coda.

STRUTTURE DATI ELEMENTARI: PILE E CODE

Per quanto concerne la pila possiamo definire gli inserimenti (push), aggiungendo elementi alla fine della sequenza, mentre le cancellazioni (pop), rimuovendo sempre l'ultimo elemento. Gli accessi avvengono quindi ad una sola estremità della sequenza di elementi, e nessun elemento interno può essere estratto prima che tutti quelli che lo seguono siano stati estratti.

In una coda, gli inserimenti (enqueue) aggiungono elementi alla fine della sequenza, mentre le cancellazioni (dequeue) ne rimuovono sempre il primo elemento. In una coda, gli accessi avvengono quindi ad entrambe le estremità della sequenza di elementi, e nessun elemento interno può essere estratto prima che tutti quelli che lo precedono siano stati estratti.

STRUTTURE DATI ELEMENTARI: PILE E CODE

```
# Creiamo una pila o stack
stack = []

# Operazione PUSH:
# aggiunge un elemento in cima allo stack
stack.append(10)
stack.append(20)
stack.append(30)

print("Stack dopo le operazioni push:", stack)

# Operazione POP:
# rimuove e restituisce l'elemento in cima allo stack
elemento_rimosso = stack.pop()

print("Elemento rimosso con pop:", elemento_rimosso)
print("Stack dopo la pop:", stack)
```

STRUTTURE DATI ELEMENTARI: PILE E CODE

```
def push(stack, elemento):
    """
    Aggiunge un elemento in cima allo stack.
    """
    stack.append(elemento)

def pop(stack):
    """
    Rimuove e restituisce l'elemento in cima allo stack.
    Se lo stack è vuoto, restituisce None.
    """
    if len(stack) == 0:
        print("Errore: lo stack è vuoto")
        return None

    return stack.pop()

# Esempio di utilizzo
pila = []

push(pila, "A")
push(pila, "B")
push(pila, "C")

print("Pila:", pila)

print("Elemento estratto:", pop(pila))
print("Pila dopo pop:", pila)
```

STRUTTURE DATI ELEMENTARI: PILE E CODE

```
# Creiamo una coda o queue
queue = []

# Operazione ENQUEUE:
# aggiunge un elemento alla fine della coda
queue.append("Mario")
queue.append("Anna")
queue.append("Luca")

print("Coda dopo le operazioni enqueue:", queue)

# Operazione DEQUEUE:
# rimuove e restituisce il primo elemento della coda
persona_servita = queue.pop(0)

print("Persona servita con dequeue:", persona_servita)
print("Coda dopo la dequeue:", queue)
```

STRUTTURE DATI ELEMENTARI: PILE E CODE

```
from collections import deque

# Creiamo una coda efficiente
coda = deque()

# ENQUEUE:
# inserisce un elemento in fondo alla coda
coda.append("Cliente 1")
coda.append("Cliente 2")
coda.append("Cliente 3")

print("Coda dopo enqueue:", coda)

# DEQUEUE:
# rimuove e restituisce il primo elemento della coda
cliente_servito = coda.popleft()

print("Cliente servito:", cliente_servito)
print("Coda dopo dequeue:", coda)
```

STRUTTURE DATI ELEMENTARI: ALBERI

In molte situazioni reali trattiamo **collezioni di oggetti** su cui sono definite in modo naturale delle **relazioni gerarchiche**. Ad esempio, ogni persona ha una relazione di discendenza diretta con i suoi due genitori, i quali a loro volta sono in relazione con i propri genitori, e così via. Un **albero genealogico** è un tipico esempio di collezione di oggetti su cui sono definite delle relazioni gerarchiche.

Un **albero (radicato)** è una coppia $T = (N,A)$ costituita da insieme N di nodi e da un insieme $A \subseteq N \times N$ di coppie di nodi, dette archi. Un **albero** può essere visto anche come un particolare tipo di **grafo**. In un albero, ogni nodo v (tranne la radice) ha un solo genitore (o padre) u tale che $(u,v) \in A$.

STRUTTURE DATI ELEMENTARI: ALBERI

Un nodo u può avere zero o più figli v tali che (u,v) ∈ A, e il loro numero viene chiamato **grado del nodo**. Un nodo senza figli è chiamato **foglia**, mentre nodi che non sono nè foglie nè la radice sono chiamati **nodi interni**. Parleremo inoltre di **antenati** e **discendenti** di un nodo intendendo i nodi raggiungibili da quel nodo salendo di padre in padre o scendendo di figlio in figlio nell'albero, rispettivamente. La **profondità (o livello)** di un nodo è il **numero di archi** che bisogna attraversare per raggiungerlo a **partire dalla radice**. Più formalmente, essa può essere definita ricorsivamente come segue:

- La radice ha profondità zero;
- Se un nodo ha profondità k, tutti i suoi figli hanno profondità k+1.

STRUTTURE DATI ELEMENTARI: ALBERI

Nodi con lo stesso genitore vengono chiamati **fratelli**, ed essi avranno quindi la stessa profondità. L'**altezza** di un albero è la **massima profondità** a cui si trova una foglia.

Particolare rilievo hanno **alcune classi di alberi** con *vincoli strutturali*, che ne facilitano la rappresentazione o consentono di realizzare operazioni su collezioni di elementi in modo efficiente. I principali vincoli strutturali riguardano il **grado** dei nodi e la **profondità**. In particolare, un albero d-ario è un albero in cui tutti i nodi tranne le foglie hanno grado d. Per d=2 diremo che **l'albero è binario**. Infine, diremo che un albero d-ario in cui tutte le foglie sono sullo stesso livello è *completo*.

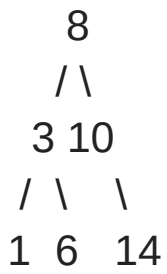
STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

Un BST, cioè **Binary Search Tree**, è un albero binario in cui ogni nodo rispetta una proprietà fondamentale:

Per ogni nodo:

- tutti i valori nel sottoalbero sinistro sono minori del nodo;
- tutti i valori nel sottoalbero destro sono maggiori del nodo.

Esempio:



Nel nodo 8, i valori 3, 1, 6 stanno a sinistra perché sono minori di 8, mentre 10 e 14 stanno a destra perché sono maggiori.

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

Operazioni principali in un BST

Le operazioni fondamentali sono:

Operazione	Descrizione
Ricerca	cerca un valore partendo dalla radice
Inserimento	aggiunge un nuovo nodo rispettando la proprietà BST
Cancellazione	rimuove un nodo mantenendo valida la struttura
Visita inorder	restituisce i valori in ordine crescente

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

```
class Nodo:
    def __init__(self, valore):
        # Valore contenuto nel nodo
        self.valore = valore

        # Figlio sinistro
        self.sinistro = None

        # Figlio destro
        self.destro = None
```

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

```python
def ricerca_bst(radice, valore):
    """
    Cerca un valore in un albero binario di ricerca.

    Ritorna True se il valore è presente, False altrimenti.
    """

    # Se il nodo è None, il valore non è stato trovato
    if radice is None:
        return False

    # Se il valore cercato è uguale al valore del nodo corrente
    if valore == radice.valore:
        return True

    # Se il valore cercato è minore, cerchiamo nel sottoalbero sinistro
    elif valore < radice.valore:
        return ricerca_bst(radice.sinistro, valore)

    # Se il valore cercato è maggiore, cerchiamo nel sottoalbero destro
    else:
        return ricerca_bst(radice.destro, valore)
```

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

```
def inserisci_bst(radice, valore):
    """
    Inserisce un valore in un BST e restituisce la radice aggiornata.
    """

    # Se l'albero è vuoto, creiamo un nuovo nodo
    if radice is None:
        return Nodo(valore)

    # Se il valore è minore, inseriamo a sinistra
    if valore < radice.valore:
        radice.sinistro = inserisci_bst(radice.sinistro, valore)

    # Se il valore è maggiore, inseriamo a destra
    elif valore > radice.valore:
        radice.destro = inserisci_bst(radice.destro, valore)

    # Se il valore è uguale, in questo esempio non inseriamo duplicati
    return radice
```

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

```
def visita_inorder(radice):
    """
    Visita inorder:
    sinistra -> radice -> destra
    """

    if radice is not None:
        visita_inorder(radice.sinistro)
        print(radice.valore, end=" ")
        visita_inorder(radice.destro)
```

STRUTTURE DATI ELEMENTARI: ALBERI DI RICERCA BINARI (BST)

```
radice = None

for valore in [8, 3, 10, 1, 6, 14]:
    radice = inserisci_bst(radice, valore)

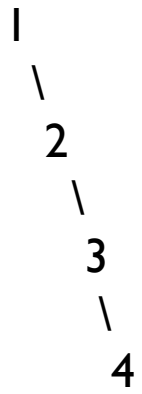
visita_inorder(radice)
```

STRUTTURE DATI ELEMENTARI: COMPLESSITA' ALBERI DI RICERCA BINARI (BST)

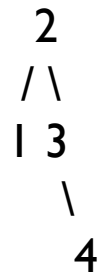
Caso	Complessità ricerca/inserimento/cancellazione
Albero bilanciato	O(log n)
Albero sbilanciato	O(n)

STRUTTURE DATI ELEMENTARI: ALBERI BILANCIATI

Gli alberi bilanciati sono alberi di ricerca che mantengono controllata la propria altezza.
Lo scopo è evitare che l'albero degeneri in una lista.
BST sbilanciato:



BST bilanciato:



STRUTTURE DATI ELEMENTARI: ALBERI BILANCIATI

Un albero bilanciato permette operazioni efficienti, tipicamente:

- ricerca: O(log n)
- inserimento: O(log n)
- cancellazione: O(log n)

STRUTTURE DATI ELEMENTARI: ALBERI AVL

Un albero AVL è un BST autobilanciante.

La regola principale degli AVL riguarda il fattore di bilanciamento.

fattore di bilanciamento = altezza(sottoalbero sinistro) - altezza(sottoalbero destro)

In un AVL, per ogni nodo deve valere:
fattore di bilanciamento $\in \{-1, 0, +1\}$

Esempio bilanciato:

```
  10
  / \
 5 15
```

Esempio non bilanciato:

```
10
\
20
\
```

STRUTTURE DATI ELEMENTARI: ALBERI AVL

Esempio non bilanciato:

```
10
  \
  20
    \
    30
```

In questo caso serve una rotazione.

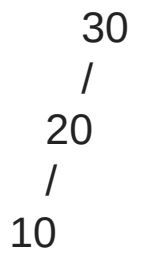
Le rotazioni servono a ristabilire il bilanciamento.
I casi principali sono:

Caso	Tipo di rotazione
Sinistra-Sinistra	rotazione a destra
Destra-Destra	rotazione a sinistra
Sinistra-Destra	rotazione sinistra + destra
Destra-Sinistra	rotazione destra + sinistra

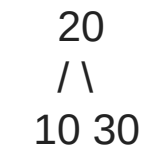
STRUTTURE DATI ELEMENTARI: ALBERI AVL

Esempio: rotazione a destra

Prima:



Dopo:



STRUTTURE DATI ELEMENTARI: ALBERI AVL

Gli AVL sono ottimi quando si fanno molte ricerche e meno modifiche. Queste le caratteristiche:

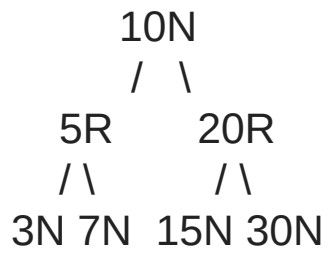
Aspetto	Albero AVL
Tipo	BST autobilanciante
Bilanciamento	molto rigido
Altezza	molto controllata
Ricerca	molto efficiente
Inserimento	può richiedere rotazioni
Cancellazione	può richiedere più rotazioni

STRUTTURE DATI ELEMENTARI: ALBERI ROSSO-NERI

Un Red-Black Tree, o albero rosso-nero, è un altro tipo di BST autobilanciante. Ogni nodo ha un colore: **rosso** oppure **nero**

Le regole principali sono:
1. Ogni nodo è rosso o nero.
2. La radice è nera.
3. Le foglie NIL sono considerate nere.
4. Un nodo rosso non può avere un figlio rosso.
5. Ogni cammino da un nodo alle foglie NIL contiene lo stesso numero di nodi neri.

Queste regole garantiscono che l'albero non diventi troppo sbilanciato.
Esempio:



STRUTTURE DATI ELEMENTARI: ALBERI ROSSO-NERI

Gli alberi rosso-neri sono molto usati nelle librerie standard perché hanno un buon compromesso tra ricerca e aggiornamento. Caratteristiche:

Aspetto	Red-Black Tree
Tipo	BST autobilanciante
Bilanciamento	meno rigido degli AVL
Altezza	controllata, ma meno stretta
Inserimento	efficiente
Cancellazione	efficiente
Uso tipico	strutture associative, mappe, set

STRUTTURE DATI ELEMENTARI: CONFRONTO AVL VS ROSSO-NERI

Aspetto	AVL	Red-Black
Bilanciamento	più rigido	meno rigido
Altezza	più bassa	leggermente più alta
Ricerca	spesso più veloce	leggermente meno veloce
Inserimento	può richiedere più rotazioni	generalmente più efficiente
Cancellazione	può essere più costosa	generalmente più efficiente
Uso ideale	molte ricerche	molte modifiche
Complessità	O(log n)	O(log n)

In sintesi:
AVL: migliore quando le ricerche sono molto frequenti.

Red-Black: migliore quando ci sono molte operazioni di inserimento e cancellazione

STRUTTURE DATI ELEMENTARI: ALBERI ROSSO-NERI

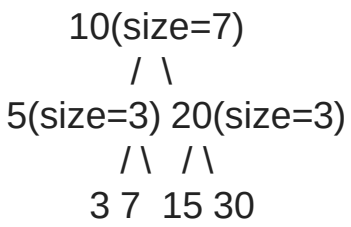
Gli alberi rosso-neri possono essere arricchiti per supportare le statistiche d'ordine.

Una statistica d'ordine permette di rispondere a domande come:

Qual è il k-esimo elemento più piccolo?
Qual è la posizione ordinata di un certo elemento?

Per farlo, ogni nodo memorizza una informazione aggiuntiva:
size = numero di nodi nel sottoalbero radicato in quel nodo

Esempio:



Con questa informazione possiamo trovare rapidamente il k-esimo elemento.

STRUTTURE DATI ELEMENTARI: HEAP

Un heap è una struttura dati ad albero, solitamente implementata con un array.

Il tipo più comune è l'heap binario.
A differenza di un BST, lo heap non mantiene un ordinamento totale tra tutti i nodi.

Mantiene invece una proprietà padre-figlio.

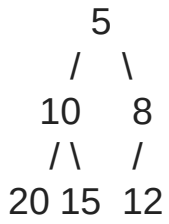
Un **heap binario** è un albero binario quasi completo.
Significa che:
- tutti i livelli sono pieni tranne eventualmente l'ultimo;
- l'ultimo livello è riempito da sinistra verso destra.

Esistono due tipi principali:
Max-heap
In un max-heap, ogni padre è maggiore o uguale dei suoi figli.

```
          50
          / \
        30 40
         / \ /
      10 20 35
```

STRUTTURE DATI ELEMENTARI: HEAP

Il massimo è sempre nella radice.
Min-heap
In un min-heap, ogni padre è minore o uguale dei suoi figli.



Il minimo è sempre nella radice.

STRUTTURE DATI ELEMENTARI: HEAP

Operazione **heapify** serve a ripristinare la proprietà dello heap.

Nel caso di un min-heap:

se un padre è maggiore di uno dei figli,
lo si scambia con il figlio più piccolo.

Proprietà	Descrizione
Forma	albero binario quasi completo
Ordinamento	padre maggiore/minore dei figli
Radice	massimo nel max-heap, minimo nel min-heap
Implementazione	spesso tramite array
Altezza	O(log n)

STRUTTURE DATI ELEMENTARI: HEAP

```
def heapify_down(heap, indice):
    """
    Ripristina la proprietà di min-heap spostando verso il basso
    l'elemento in posizione 'indice'.
    """

    n = len(heap)

    while True:
        # Calcoliamo gli indici dei figli
        sinistro = 2 * indice + 1
        destro = 2 * indice + 2

        # Supponiamo che il nodo corrente sia il più piccolo
        piu_piccolo = indice

        # Se il figlio sinistro esiste ed è più piccolo
        if sinistro < n and heap[sinistro] < heap[piu_piccolo]:
            piu_piccolo = sinistro

        # Se il figlio destro esiste ed è più piccolo
        if destro < n and heap[destro] < heap[piu_piccolo]:
            piu_piccolo = destro
```

STRUTTURE DATI ELEMENTARI: HEAP

```
# Se il nodo corrente è già più piccolo dei figli, abbiamo finito
if piu_piccolo == indice:
    break

# Scambiamo il nodo corrente con il figlio più piccolo
heap[indice], heap[piu_piccolo] = heap[piu_piccolo], heap[indice]

# Continuiamo dal nuovo indice
indice = piu_piccolo
```

STRUTTURE DATI ELEMENTARI: HEAP

```
def heapify_up(heap, indice):
    """
    Ripristina la proprietà di min-heap spostando verso l'alto
    l'elemento appena inserito.
    """

    while indice > 0:
        # Calcoliamo l'indice del padre
        padre = (indice - 1) // 2

        # Se il padre è già minore o uguale, la proprietà è rispettata
        if heap[padre] <= heap[indice]:
            break

        # Altrimenti scambiamo figlio e padre
        heap[padre], heap[indice] = heap[indice], heap[padre]

        # Continuiamo dal padre
        indice = padre
```

STRUTTURE DATI ELEMENTARI: HASH TABLE

Una Hash Table è una struttura dati usata per memorizzare coppie: chiave → valore

Esempio:
"MRARSS90A01H501Z" → "Mario Rossi"
"BNCLCU91B02F839X" → "Luca Bianchi"

Una Hash Table usa una funzione hash per trasformare una chiave in un indice dell'array.
chiave → funzione hash → indice

Esempio:
"mario" → hash("mario") → posizione 3

Quindi il dato viene salvato nella posizione calcolata.

STRUTTURE DATI ELEMENTARI: HASH TABLE

Nel caso peggiore, se ci sono molte collisioni, la complessità può diventare O(n).

Collisioni

Una collisione avviene quando due chiavi diverse producono lo stesso indice.
"mario" → indice 3
"luca" → indice 3

Per gestire le collisioni si possono usare varie tecniche, tra cui:

- chaining: ogni cella contiene una lista di elementi
- open addressing: si cerca un'altra posizione libera

Operazione	Esempio Python	Complessità media
Inserimento	`dict[chiave] = valore`	O(1)
Ricerca	`dict[chiave]`	O(1)
Cancellazione	`del dict[chiave]`	O(1)

STRUTTURE DATI ELEMENTARI: HASH TABLE

```
class HashTable:
    def __init__(self, dimensione):
        # Creiamo una lista di liste vuote
        # Ogni posizione può contenere più coppie chiave-valore
        self.dimensione = dimensione
        self.tabella = [[] for _ in range(dimensione)]

    def _hash(self, chiave):
        # Calcola un indice usando la funzione hash di Python
        return hash(chiave) % self.dimensione

    def inserisci(self, chiave, valore):
        # Calcoliamo l'indice della chiave
        indice = self._hash(chiave)

        # Controlliamo se la chiave esiste già
        for coppia in self.tabella[indice]:
            if coppia[0] == chiave:
                coppia[1] = valore
                return

        # Se la chiave non esiste, aggiungiamo una nuova coppia
        self.tabella[indice].append([chiave, valore])
```

STRUTTURE DATI ELEMENTARI: HASH TABLE

```
def cerca(self, chiave):
    # Calcoliamo l'indice della chiave
    indice = self._hash(chiave)

    # Cerchiamo la chiave nella lista associata all'indice
    for coppia in self.tabella[indice]:
        if coppia[0] == chiave:
            return coppia[1]

    # Se la chiave non viene trovata
    return None

def cancella(self, chiave):
    # Calcoliamo l'indice della chiave
    indice = self._hash(chiave)

    # Cerchiamo la chiave e la rimuoviamo
    for i, coppia in enumerate(self.tabella[indice]):
        if coppia[0] == chiave:
            del self.tabella[indice][i]
            return True

    return False
```

STRUTTURE DATI ELEMENTARI: HASH TABLE

```
# Esempio di utilizzo
ht = HashTable(5)

ht.inserisci("A001", "Mario Rossi")
ht.inserisci("A002", "Luca Bianchi")
ht.inserisci("A003", "Anna Verdi")

print(ht.cerca("A002"))

ht.cancella("A003")

print(ht.cerca("A003"))
```

STRUTTURE DATI ELEMENTARI: HEAP VS AVL

Aspetto	Heap	AVL
Struttura	albero quasi completo	BST bilanciato
Ordinamento	solo padre-figlio	ordinamento totale sinistra-radice-destra
Ricerca generica	inefficiente, O(n)	efficiente, O(log n)
Minimo/massimo	molto efficiente, O(1)	O(log n), oppure O(1) se mantenuto puntatore
Inserimento	O(log n)	O(log n)
Cancellazione	tipicamente radice, O(log n)	qualsiasi nodo, O(log n)
Uso tipico	priority queue, heapsort	dizionari ordinati, insiemi ordinati

STRUTTURE DATI ELEMENTARI: HASH TABLE VS ALBERI BILANCIATI

Aspetto	Hash Table	Alberi bilanciati
Ricerca media	O(1)	O(log n)
Ricerca peggiore	O(n)	O(log n)
Ordinamento	non mantiene ordine	mantiene ordine
Minimo/massimo	non efficiente	efficiente
Range query	inefficiente	efficiente
Successore/predecessore	inefficiente	efficiente
Dipendenza da funzione hash	sì	no
Uso ideale	accesso rapido per chiave	dati ordinati e interrogazioni su intervalli

STRUTTURE DATI ELEMENTARI: PROS E CONS

Struttura	Punto di forza	Punto debole	Complessità tipica
BST semplice	facile da implementare	può sbilanciarsi	O(n) nel caso peggiore
AVL	ricerche molto efficienti	aggiornamenti più costosi	O(log n)
Red-Black	buon compromesso generale	ricerca leggermente meno ottimizzata di AVL	O(log n)
Heap	ottimo per priorità	ricerca generica inefficiente	O(log n) inserimento/estrazione
Hash Table	accesso medio molto veloce	non mantiene ordine	O(1) medio

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

